10/10

Event-Driven Notification Engine

The Complete 10/10 Build Guide

Production-Grade • Security-Hardened • Load-Tested • Multi-Tenant

NestJS TypeScript RabbitMQ BullMQ MongoDB Redis Socket.io Docker K8s k6

Mohamed — Backend Developer

Portfolio Project • 2025

Table of Contents

■ = New phases that elevate this project from 9/10 to 10/10

Project Overview

This guide walks you through building a **production-grade, event-driven notification microservice** from scratch using NestJS and TypeScript. The system receives domain events (like order.placed, payment.failed, user.signup) and orchestrates multi-channel notification delivery across email, SMS, push notifications, and in-app messages.

The **10/10 version** goes beyond core functionality to include security hardening, multi-tenancy, load testing with documented benchmarks, schema registry, CI/CD pipeline, cloud deployment with Infrastructure-as-Code, and Architecture Decision Records — everything a senior engineer or staff-level candidate would implement.

■■ ARCHITECTURE INSIGHT

High-Level Architecture: Business services publish domain events to RabbitMQ. The notification engine consumes events, validates schemas, deduplicates, checks user preferences, resolves templates, and dispatches to isolated per-channel BullMQ queues. Each channel processes independently with its own retry logic, circuit breakers, and fallback strategies. Delivery results are tracked in MongoDB with real-time metrics exposed via Prometheus and a live dashboard via Socket.io.

```
[Business Services] → publish → [RabbitMQ Topic Exchange]
   ↓
[Event Consumer] → schema validate → deduplicate → [Event Processor]
   ↓
[User Prefs Check] → [Template Engine + Redis Cache] → [Channel Router]
   ↓          ↓           ↓          ↓
[Email Q]   [SMS Q]    [Push Q]   [In-App Q]
   ↓          ↓           ↓          ↓
[SendGrid]  [Twilio]   [FCM]     [MongoDB + Socket.io]
   ↓ circuit breaker + fallback routing ↓
[Delivery Tracking] → [Prometheus Metrics] → [Grafana Dashboard]
```

Prerequisites

- Solid JavaScript/TypeScript fundamentals
- Basic Node.js, npm, and REST API knowledge
- Basic database understanding (SQL or NoSQL)
- Git version control, Docker Desktop installed
- VS Code with ESLint + Prettier extensions

PHASE 1: Foundation & Project Setup

Step 1 — NestJS & TypeScript Foundation

■ TOPICS TO STUDY FIRST

■ **TypeScript Deep Dive:** Interfaces, generics, enums, decorators, utility types (Partial, Pick, Omit), type guards, strict mode

■ **NestJS Architecture:** Modules, Controllers, Providers, Dependency Injection, @Module decorator (imports, exports, providers)

■ **NestJS Decorators:** @Injectable(), @Controller(), @Get/@Post/@Put/@Delete, custom decorators, @Body/@Param/@Query

■ **Configuration:** @nestjs/config, .env files, Joi schema validation, ConfigService injection, environment-specific configs

■ **NestJS CLI:** nest new, nest generate, monorepo mode, library generation

■ **Project Structure:** Feature-based module organization, shared/common modules, barrel exports, path aliases

■ WHAT TO BUILD

✓ Scaffold project: nest new notification-engine with strict TypeScript (strict: true, strictNullChecks: true)

✓ Set up @nestjs/config with .env file and Joi validation schema for all environment variables

✓ Create module structure: CoreModule, SharedModule, ConfigModule with proper imports/exports

✓ Define global exception filters (HttpExceptionFilter) and interceptors (LoggingInterceptor, TransformInterceptor)

✓ Set up path aliases: @shared/*, @modules/*, @common/* in tsconfig.json

✓ Create health-check endpoint (GET /health) with system status, uptime, and memory usage

✓ Configure global ValidationPipe with class-validator + class-transformer, whitelist: true, forbidNonWhitelisted: true

✓ Set global API prefix (/api/v1), enable CORS, configure Helmet for security headers

✓ Set up ESLint + Prettier with strict rules, husky pre-commit hooks for lint + format

■ PRO TIP

Enable **strict: true** in tsconfig.json from day one. It catches null/undefined bugs early — critical in a notification system where missing data causes silent failures. Also set up **husky + lint-staged** for pre-commit quality gates.

Step 2 — MongoDB & Mongoose Integration

■ TOPICS TO STUDY FIRST

■ **MongoDB Fundamentals:** Documents, collections, BSON, ObjectId, embedded vs referenced documents, when to embed vs reference

■ **Mongoose + NestJS:** @nestjs/mongoose, MongooseModule.forRoot/forFeature, @Schema/@Prop decorators, SchemaFactory

■ **Schema Design:** Compound indexes, unique indexes, TTL indexes, text indexes, virtuals, pre/post hooks, timestamps

■ **Queries:** find, aggregate, populate, lean(), cursor-based pagination, projection, query optimization with explain()

■ **Data Modeling:** Notification, EventLog, Template, UserPreference, DeliveryEvent, Tenant schemas

■ WHAT TO BUILD

✓ Install @nestjs/mongoose + mongoose, configure MongooseModule.forRootAsync with ConfigService

✓ Design Notification schema: userId, tenantId, channel, event, status (pending/queued/sent/failed/cancelled), payload, metadata, retryCount, sentAt, timestamps

✓ Design EventLog schema: eventType, eventId (unique for dedup), payload, processedAt, status, correlationId, tenantId

✓ Design NotificationTemplate schema: name, channel, subject, body, variables[], version, isActive, tenantId

✓ Design UserPreference schema: userId, tenantId, channels (per-channel opt-in/out with contact info), quietHours (start, end, timezone), mutedEvents[], globalOptOut

✓ Add compound indexes: { userId: 1, createdAt: -1 }, { eventId: 1 } (unique), { status: 1, channel: 1 }, { tenantId: 1, userId: 1 }

✓ Add TTL index on EventLog (auto-cleanup after 30 days), TTL index on processed notifications

✓ Create Repository pattern classes wrapping Mongoose models for testability and abstraction

✓ Write seed scripts for test templates and sample user preferences

✓ Implement soft-delete with deletedAt field and default query scoping to exclude deleted records

■ PRO TIP

Design schemas with **query patterns in mind**. Use **lean()** when you don't need Mongoose document methods — it returns plain objects and is 5-10x faster. Add the **tenantId** field now even if multi-tenancy comes later; retrofitting is painful.

PHASE 2: Event Ingestion & Message Broker

Step 3 — RabbitMQ Integration & Event Ingestion

■ TOPICS TO STUDY FIRST

■ **Message Broker Fundamentals:** Producers, consumers, queues, exchanges, bindings, ack/nack, dead letter queues (DLQ)

■ **RabbitMQ:** Exchange types (direct, topic, fanout, headers), routing keys, prefetch, durable queues, persistent messages

■ **AMQP Protocol:** Channels, connections, virtual hosts, message properties (headers, correlation-id, reply-to)

■ **NestJS Microservices:** @nestjs/microservices, RabbitMQ transport, ClientProxy, @EventPattern decorator

■ **Event-Driven Architecture:** Domain events, eventual consistency, event schemas, decoupling principles

■ **Docker Compose:** Multi-container setup, depends_on with healthchecks, volumes, networks, env files

■ WHAT TO BUILD

✓ Create docker-compose.yml with RabbitMQ (management :15672), MongoDB (replica set), and Redis

✓ Install @nestjs/microservices + amqplib/amqp-connection-manager, configure with auto-reconnection

✓ Create topic exchange 'notification.events' with durable: true

✓ Define routing key pattern: domain.action (order.placed, payment.failed, user.signup)

✓ Create EventConsumerService: listen to exchange, route events to processors

✓ Implement ack on success, nack + requeue on transient failure, nack + DLQ on permanent failure

✓ Create Dead Letter Queue with DLQ exchange for failed events after max retries

✓ Build event publisher REST endpoint for testing: POST /api/v1/events/publish

✓ Set prefetch count = 10 (tune later based on load testing results)

✓ Add connection event logging: connected, disconnected, reconnecting with timestamps

■■ ARCHITECTURE INSIGHT

Why RabbitMQ + BullMQ together? RabbitMQ handles external event ingestion — reliable inter-service communication with routing and DLQs. BullMQ (Redis-based) handles internal job processing — delayed jobs, rate limiting, priorities, per-channel isolation. RabbitMQ = external message bus. BullMQ = internal job scheduler.

Step 4 — Event Processing & Idempotency Layer

■ TOPICS TO STUDY FIRST

■ **Idempotency:** Why it matters in distributed systems, idempotency keys, at-least-once vs exactly-once semantics

■ **Event Deduplication:** Dedup key strategies (event ID, payload hash), time windows, two-tier dedup (Redis + MongoDB)

■ **Schema Validation:** class-validator, DTOs for events, versioned schemas, backward compatibility rules

■ **Event Versioning:** Additive-only changes, deprecation strategy, version field, schema evolution patterns

■ **Design Patterns:** Strategy (event processors), Chain of Responsibility (validation), Factory (handler resolution)

■ WHAT TO BUILD

✓ Define base event DTO: { eventId, eventType, version, timestamp, payload, metadata: { correlationId, source, tenantId } }

✓ Create specific DTOs: OrderPlacedEvent, PaymentFailedEvent, UserSignupEvent with validated payloads

✓ Implement two-tier dedup: Redis fast check (SET eventId NX EX 3600) + MongoDB unique index fallback

✓ Build EventValidationPipe: validate schema, check version compatibility, sanitize payload

✓ Create EventProcessorFactory: maps eventType to handler using Strategy pattern

✓ Implement EventOrchestrator: dedup → validate → determine channels → check preferences → enqueue to BullMQ

✓ Log every event with correlationId to EventLog collection for full audit trail

✓ Handle unknown event types: log warning, send to DLQ, never crash

✓ Write unit tests for deduplication and validation logic

■ PRO TIP

Two-tier deduplication is a production pattern: Redis SET NX catches recent duplicates in microseconds, MongoDB unique index is the durable fallback. Speed + reliability.

PHASE 3: Multi-Channel Delivery System

Step 5 — BullMQ Queue Architecture

■ TOPICS TO STUDY FIRST

■ **BullMQ:** Queues, workers, job lifecycle (waiting→active→completed/failed), job options (attempts, backoff, delay, priority)

■ **Redis as Queue Backend:** Data structures, persistence (RDB/AOF), connection pooling, ioredis config

■ **Queue Patterns:** Per-channel isolation, named queues, queue events, rate limiting, job batching

■ **Worker Config:** Concurrency, lock duration, stalled job recovery, sandboxed processors

■ **NestJS + BullMQ:** @nestjs/bullmq, registerQueue, @Processor, @Process, queue event decorators

■ WHAT TO BUILD

✓ Install @nestjs/bullmq + bullmq + ioredis, configure with Redis connection via ConfigService

✓ Create isolated queues: 'notifications:email', 'notifications:sms', 'notifications:push', 'notifications:in-app'

✓ Implement ChannelRouter: dispatches to appropriate queues based on event→channel mapping

✓ Define job data interface: { notificationId, userId, tenantId, channel, event, templateId, variables, priority, metadata }

✓ Configure per-queue defaults: email (3 attempts, exponential), SMS (5 attempts, fixed), push (3), in-app (1)

✓ Add job priority: CRITICAL=1, HIGH=2, NORMAL=3, LOW=4

✓ Set up bull-board for queue monitoring in development

✓ Configure stalled job recovery (stalledInterval, maxStalledCount)

✓ Implement queue event listeners: log active, completed, failed, stalled events with correlationId

■■ ARCHITECTURE INSIGHT

Why separate queues per channel? Fault isolation. If email provider goes down and the email queue backs up with 10K jobs, SMS and push continue normally. Each queue gets independent concurrency, retry policies, and rate limits.

Step 6 — Channel Providers Implementation

■ TOPICS TO STUDY FIRST

■ **Email:** SMTP, transactional email services (SendGrid, SES), nodemailer, HTML vs plain text, MJML templates

■ **SMS:** Twilio SDK, E.164 phone format, sender IDs, character limits, message segments

■ **Push:** Firebase Cloud Messaging (FCM), device token management, notification payloads, topic messaging

■ **Provider Abstraction:** Adapter pattern, interface-based design, provider factory, swappable implementations

■ **Error Handling:** Provider-specific error codes, transient vs permanent failures, error classification

■ WHAT TO BUILD

✓ Define IChannelProvider interface: send(notification): Promise<DeliveryResult>

✓ Define DeliveryResult: { success, providerId, externalId?, error?, latencyMs }

✓ Implement EmailProvider (nodemailer + SendGrid / Ethereal for dev)

✓ Implement SmsProvider (Twilio SDK with error code handling)

✓ Implement PushProvider (Firebase Admin SDK with device token validation)

✓ Implement InAppProvider: save to MongoDB + emit Socket.io event

✓ Create ProviderFactory: resolves channel name to provider instance via NestJS DI

✓ Create BullMQ Worker per channel queue calling appropriate provider

✓ Classify errors: TRANSIENT (retry), PERMANENT (fail+DLQ), RATE_LIMITED (retry with delay)

✓ Log every delivery attempt: channel, provider, latency, success/failure, correlationId

✓ Create mock provider mode (env flag) for development — logs instead of sending

■ PRO TIP

Use **Ethereal (ethereal.email)** for email testing — captures emails in a web UI without sending. Twilio has free test credentials. Always have a **DRY_RUN** env flag.

PHASE 4: Resilience & Fault Tolerance

Step 7 — Retry Mechanisms & Exponential Backoff

■ TOPICS TO STUDY FIRST

■ **Retry Strategies:** Fixed, exponential, exponential+jitter, linear backoff — when to use each

■ **Backoff Math:** delay = baseDelay × 2^attempt + random_jitter. Why jitter prevents thundering herd

■ **BullMQ Retries:** attempts, backoff config, custom backoff functions

■ **Dead Letter Queues:** When to DLQ, DLQ monitoring, replay strategies

■ **Failure Classification:** HTTP 4xx=permanent, 5xx=transient, 429=rate limited, network errors, timeouts

■ WHAT TO BUILD

✓ Configure BullMQ exponential backoff: { type: 'exponential', delay: 2000 } (2s, 4s, 8s, 16s...)

✓ Add jitter: custom backoff function adding random 0-1000ms to prevent thundering herd

✓ Per-channel retry limits: email=3, SMS=5, push=3, in-app=1

✓ Create RetryClassifier: error → RETRY / FAIL / DELAY_RETRY with specific delay

✓ Handle HTTP 429: extract Retry-After header, use BullMQ delayed jobs

✓ Implement DLQ: failed jobs move to 'notifications:dlq' with full error history

✓ Build DLQ monitor: periodic count check, alert if threshold exceeded

✓ Add retry metadata to notification records: attemptNumber, lastError, nextRetryAt

✓ Create manual retry endpoint: POST /api/v1/notifications/:id/retry

✓ Create DLQ replay endpoint: POST /api/v1/dlq/replay

Step 8 — Circuit Breaker Pattern

■ TOPICS TO STUDY FIRST

■ **Circuit Breaker:** States (CLOSED→OPEN→HALF_OPEN), failure threshold, reset timeout, recovery

■ **Why Circuit Breakers:** Prevent cascading failures, fail fast, give services recovery time

■ **opossum Library:** Node.js circuit breaker, configuration, event handling, fallback functions

■ **Fallback Strategies:** Alternative provider, queue for later, graceful degradation, ops notification

■ **Bulkhead Pattern:** Isolating resources per provider to prevent one slow provider from exhausting all resources

■ WHAT TO BUILD

✓ Install opossum, create CircuitBreakerWrapper for each provider's send() method

✓ Configure per-provider: failureThreshold=5, resetTimeout=30s, halfOpenRequests=3

✓ OPEN state: immediately reject with CircuitOpenError, skip provider call

✓ HALF_OPEN state: limited requests, track success/failure ratio for recovery decision

✓ Add circuit breaker event logging: state changes, fallback triggers, recovery

✓ Implement fallback routing: primary fails → try backup provider (SendGrid→SES→Mailgun)

✓ Provider health endpoint: GET /api/v1/health/providers returning all circuit breaker states

✓ Bulkhead: limit concurrent requests per provider (max 50 concurrent Twilio calls)

✓ Write integration tests: simulate failure → verify circuit opens → verify recovery

■■ ARCHITECTURE INSIGHT

Full resilience flow: Provider call fails → RetryClassifier decides retry/fail → if multiple failures, circuit breaker trips OPEN → subsequent requests fail fast → after reset timeout, HALF_OPEN tests recovery → if success, close circuit → fallback provider handles urgent notifications meanwhile.

PHASE 5: Template Engine & User Preferences

Step 9 — Template Engine with Redis Caching

■ TOPICS TO STUDY FIRST

■ **Template Interpolation:** {{variable}} substitution, Handlebars basics, custom syntax, HTML escaping

■ **Redis:** Data types, TTL, key naming, connection pooling, cache-aside pattern, cache stampede prevention

■ **Caching Strategies:** Cache-aside, write-through, invalidation, TTL-based expiry

■ **Template Versioning:** Version field, active/inactive, A/B testing, fallback to previous version

■ WHAT TO BUILD

✓ Configure ioredis with ConfigService, implement TemplateService with CRUD

✓ Build TemplateEngine: interpolate {{variable}} with regex or Handlebars, support nested {{user.firstName}}

✓ Template validation: verify all required variables present before rendering

✓ Redis caching: key 'template:{name}:{channel}:{version}' with 1-hour TTL

✓ Cache invalidation on template update, stampede prevention with SETNX lock

✓ Template management API: POST/GET/PUT/DELETE /api/v1/templates

✓ Preview endpoint: POST /api/v1/templates/preview — render without sending

✓ Seed default templates: welcome email, order confirmation, password reset, payment failed

Step 10 — User Preference Engine

■ TOPICS TO STUDY FIRST

■ **Timezone Handling:** UTC storage, conversion (luxon/date-fns-tz), IANA identifiers, DST edge cases

■ **Preference Schema:** Per-channel opt-in/out, per-event muting, quiet hours, priority overrides

■ **Priority System:** CRITICAL/HIGH/NORMAL/LOW, which preferences CRITICAL can override

■ **Compliance:** CAN-SPAM, GDPR consent, unsubscribe mechanisms, audit trail

■ WHAT TO BUILD

✓ Design UserPreference: channels with per-channel enable/contact, quietHours with timezone, mutedEvents[], priority overrides

✓ PreferenceService: CRUD + getEffectivePreferences() merging defaults with user overrides

✓ PreferenceFilter: check if notification should be sent before enqueuing

✓ Quiet hours: convert current time to user timezone, handle overnight spans (22:00-08:00)

✓ Priority override: CRITICAL notifications bypass quiet hours and muted events

✓ Per-event muting: mute marketing.promo but keep order.shipped

✓ Preference API: GET/PATCH /api/v1/users/:userId/preferences

✓ One-click unsubscribe endpoint with token-based verification (email compliance)

✓ Cache preferences in Redis with 10-min TTL, invalidate on update

✓ Log preference decisions: which notifications filtered and why

PHASE 6: Digest, Batching & Rate Limiting

Step 11 — Digest & Batching System

■ TOPICS TO STUDY FIRST

■ **Notification Digests:** Aggregating notifications into summaries, digest strategies

■ **NestJS Cron:** @nestjs/schedule, @Cron decorator, cron expressions, handling overlapping executions

■ **Distributed Locking:** Redis SETNX + TTL, Redlock algorithm, why locks matter in multi-instance

■ **MongoDB Aggregation:** $group, $match, $sort for grouping by user + time window

■ WHAT TO BUILD

✓ Install @nestjs/schedule, create DigestService for collecting pending notifications

✓ Hourly digest cron: query pending, group by userId, render digest template, send single email

✓ Daily digest cron: 24-hour window, send at 9am user's local time

✓ Redis distributed lock: acquire 'digest:hourly:lock' with SET NX EX before processing

✓ Handle lock contention: skip if another instance holds the lock

✓ Mark notifications as 'digested' to prevent re-inclusion

✓ Digest preference per user: IMMEDIATE / HOURLY / DAILY / WEEKLY

✓ Manual trigger: POST /api/v1/digest/trigger for testing

Step 12 — Rate Limiting & Fallback Routing

■ TOPICS TO STUDY FIRST

■ **Rate Limiting:** Token bucket, sliding window, fixed window, leaky bucket algorithms

■ **BullMQ Rate Limiting:** Built-in limiter { max, duration }, group-based limiting

■ **Provider Limits:** Twilio SMS/second, SendGrid daily limits, FCM per-device limits

■ **Fallback Routing:** Provider chain, health-based routing, weighted routing

■ WHAT TO BUILD

✓ Per-channel BullMQ rate limiting: email (100/s), SMS (10/s), push (500/s)
✓ Redis per-user rate limiter: max 20 notifications/hour (excluding CRITICAL)
✓ Sliding window using Redis sorted sets for precise limiting
✓ FallbackRouter: ordered provider list, skip circuit-OPEN providers
✓ Health-based routing: prefer CLOSED > HALF_OPEN, never route to OPEN
✓ Rate limit headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset
✓ Handle provider 429: pause queue for Retry-After duration
✓ Rate limit monitoring: track throttled notifications per channel per minute

PHASE 7: Observability & Monitoring

Step 13 — Structured Logging & Correlation IDs

■ TOPICS TO STUDY FIRST

■ **Structured Logging:** JSON logs, log levels, log context, why structured > plain text

■ **Pino:** High-performance Node.js logger, transports, nestjs-pino integration

■ **Correlation IDs:** Request tracing, UUID generation, X-Correlation-ID header, AsyncLocalStorage

■ **AsyncLocalStorage:** Request-scoped context without passing correlationId through every function

■ WHAT TO BUILD

✓ Install nestjs-pino, configure log levels per environment (dev=debug, prod=warn)

✓ CorrelationIdMiddleware: extract/generate X-Correlation-ID, store in AsyncLocalStorage

✓ LoggerService wrapper: auto-include correlationId, userId, channel, tenantId in every log

✓ Request logging interceptor: method, url, statusCode, responseTime

✓ Full lifecycle logging: event_received → preferences_checked → template_rendered → queued → delivered/failed

✓ Error logging: stack trace, sanitized payload (no PII), correlationId

✓ Log redaction: auto-mask emails, phone numbers, tokens in all log output

✓ Configure retention: 7d debug, 30d info, 90d error

Step 14 — Metrics, Analytics & Dashboard

■ TOPICS TO STUDY FIRST

■ **Prometheus:** Pull-based metrics, counters/gauges/histograms, labels, /metrics endpoint

■ **Grafana:** Dashboard creation, panels, alerting, Prometheus data source

■ **Key Metrics:** Delivery rate, failure rate, latency (p50/p95/p99), queue depth, circuit state

■ **Alerting:** Threshold rules, notification channels, severity levels

■ WHAT TO BUILD

✓ Install prom-client, expose /metrics endpoint

✓ Counters: notifications_sent_total{channel,status}, events_received_total{type}, retries_total{channel}

✓ Histograms: delivery_duration_seconds{channel}, queue_wait_seconds{channel}

✓ Gauges: queue_depth{channel}, circuit_breaker_state{provider}

✓ AnalyticsService: record delivery events to MongoDB with full metadata

✓ Aggregation queries: success rate by channel (1h/24h/7d), avg latency by provider, failure breakdown

✓ Add Prometheus + Grafana to docker-compose with pre-configured dashboards

✓ Create Grafana panels: delivery rate, failure rate, queue depth, latency, circuit status

✓ Alerting: failure rate > 5%, queue depth > 500

✓ REST analytics API: GET /api/v1/analytics/summary

✓ Real-time stats via Server-Sent Events (SSE) for live dashboard

PHASE 8: Real-Time & WebSockets

Step 15 — In-App Notifications with Socket.io

■ TOPICS TO STUDY FIRST

■ **WebSocket Protocol:** Full-duplex, connection upgrade, frames, ping/pong, disconnection handling

■ **Socket.io:** Events, rooms, namespaces, auto-reconnection, fallback to long-polling

■ **NestJS Gateway:** @WebSocketGateway, @SubscribeMessage, handleConnection/handleDisconnect

■ **Scaling WebSockets:** Redis adapter for multi-instance, sticky sessions, pub/sub cross-instance

■ WHAT TO BUILD

✓ Install @nestjs/websockets + @nestjs/platform-socket.io + socket.io

✓ NotificationGateway: @WebSocketGateway with CORS and JWT auth guard

✓ JWT-based WebSocket auth: validate token during handshake, reject unauthorized

✓ Join users to personal rooms: 'user:{userId}', emit to user's room

✓ InAppProvider: save to MongoDB + emit Socket.io event to user room

✓ Events: 'notification:new', 'notification:read', 'notification:count_update'

✓ Notification feed API: GET /api/v1/notifications?cursor=X&limit;=20 (cursor-based pagination)

✓ Mark-as-read: PATCH /notifications/:id/read, PATCH /notifications/read-all

✓ Unread count in Redis, invalidated on new notification / read

✓ Add @socket.io/redis-adapter for horizontal scaling

✓ Queue in-app notifications for offline users, deliver batch on reconnection

■ PRO TIP

The **Redis adapter for Socket.io** is essential. Without it, notifications processed on Instance 2 won't reach a user connected to Instance 1. Redis pub/sub bridges all instances.

PHASE 9: Containerization

Step 16 — Docker & Docker Compose

■ TOPICS TO STUDY FIRST

■ **Docker:** Images, containers, layers, Dockerfile (FROM, COPY, RUN, CMD), .dockerignore, multi-stage builds

■ **Docker Compose:** Service definitions, depends_on + healthchecks, volumes, networks, env files

■ **Production Practices:** Non-root user, read-only fs, resource limits, log drivers, SIGTERM handling

■ WHAT TO BUILD

✓ Multi-stage Dockerfile: Stage 1 (build) install deps + compile TS; Stage 2 (prod) copy compiled JS + prod deps only

✓ Base image: node:20-alpine for minimal size

✓ .dockerignore: node_modules, dist, .git, .env, *.md, tests

✓ docker-compose.yml: app, MongoDB (replica set), Redis, RabbitMQ, Prometheus, Grafana, bull-board

✓ Service deps with healthchecks: app depends on mongo/redis/rabbitmq (healthy)

✓ MongoDB replica set init script (required for transactions)

✓ Mount Prometheus config + Grafana dashboard provisioning as volumes

✓ Graceful shutdown: app.enableShutdownHooks(), close DB, drain queues, close WebSocket

✓ Full system smoke test: docker compose up → publish event → verify delivery

PHASE 10: Testing & CI/CD Pipeline ■

■ 10/10 ADDITION — This phase elevates the project beyond typical portfolio projects

Step 17 — Comprehensive Testing Strategy

■ TOPICS TO STUDY FIRST

■ **Testing Pyramid:** Unit (fast, many) → Integration (slower, some) → E2E (slowest, few), what to test at each level

■ **NestJS Testing:** @nestjs/testing, Test.createTestingModule, overrideProvider, mock providers

■ **Mocking:** Jest mocks, jest.fn(), jest.spyOn(), mocking Twilio/SendGrid, test doubles (stubs, fakes, spies)

■ **Integration Tests:** mongodb-memory-server (in-memory Mongo), ioredis-mock, BullMQ job processing tests

■ **E2E Tests:** Supertest for HTTP, testing WebSocket connections, full event→delivery flow

■ **Code Coverage:** Istanbul/c8, coverage thresholds, branch vs line vs function coverage, coverage badges

■ WHAT TO BUILD

✓ Configure Jest with path aliases, coverage thresholds (statements: 85%, branches: 80%, functions: 85%)

✓ Unit test every critical service: EventDeduplicationService, TemplateEngine, PreferenceFilter, RetryClassifier, CircuitBreakerWrapper

✓ Unit test edge cases: quiet hours crossing midnight, DST transitions, missing template variables, malformed events, empty payloads

✓ Set up mongodb-memory-server for integration tests (in-memory MongoDB per test suite)

✓ Integration test: publish event → notification created → job enqueued → delivery recorded

✓ Integration test: dedup — publish same eventId twice → only one notification created

✓ Integration test: preference filter — publish to opted-out user → notification cancelled

✓ E2E tests with Supertest: all REST endpoints, auth, error responses, rate limiting

✓ E2E test: full flow from event publish to delivery tracking update

✓ WebSocket test: connect, receive real-time notification, verify unread count update

✓ Add coverage badge to README using Istanbul reporter + shields.io

■ 10/10 DIFFERENTIATOR

Testing is the **#1 gap** that separates 9/10 from 10/10. A coverage badge in the README, unit tests on every service, and integration tests proving the full pipeline works — this tells reviewers you ship production code, not prototypes.

Step 18 — CI/CD Pipeline with GitHub Actions

■ TOPICS TO STUDY FIRST

■ **GitHub Actions:** Workflows, jobs, steps, triggers (push, PR), secrets, environment variables, artifacts

■ **CI Pipeline Design:** Lint → Type-check → Unit test → Integration test → Build → Docker image → Push

■ **Docker in CI:** docker-compose for integration tests, service containers, caching Docker layers

■ **Branch Protection:** Required checks, PR reviews, status badges, merge requirements

■ **Semantic Versioning:** semver, conventional commits, automated changelog, release tags

■ WHAT TO BUILD

✓ Create .github/workflows/ci.yml triggered on push to main and PRs

✓ Job 1 — Lint & Type Check: run ESLint + tsc --noEmit

✓ Job 2 — Unit Tests: run Jest unit tests with coverage report, fail if below thresholds

✓ Job 3 — Integration Tests: spin up MongoDB + Redis + RabbitMQ via docker-compose, run integration suite

✓ Job 4 — Build: compile TypeScript, build Docker image, verify image starts

✓ Upload coverage report as artifact, add coverage badge to README

✓ Configure branch protection: require all CI checks to pass before merge

✓ Add conventional commit linting (commitlint + husky commit-msg hook)

✓ Create release workflow: tag → build → push Docker image to registry (GHCR or Docker Hub)

✓ Add CI status badge to README: 'Build: Passing'

■ 10/10 DIFFERENTIATOR

A green CI badge in the README instantly signals professionalism. Reviewers see lint + typecheck + tests + build running automatically and think: this person works like a senior engineer.

PHASE 11: Security Hardening ■

■ 10/10 ADDITION — This phase elevates the project beyond typical portfolio projects

Step 19 — Authentication & Event Verification

■ TOPICS TO STUDY FIRST

■ **HMAC Signatures:** Hash-based message authentication, SHA-256, webhook signature verification pattern

■ **JWT Authentication:** Access tokens, refresh tokens, @nestjs/jwt, guards, strategies (Passport)

■ **API Key Auth:** Key generation, hashing (never store plaintext), key rotation, per-tenant keys

■ **Rate Limiting API:** @nestjs/throttler, per-endpoint limits, IP-based + API-key-based throttling

■ **Input Sanitization:** XSS prevention in templates, SQL/NoSQL injection prevention, payload size limits

■ WHAT TO BUILD

✓ Implement HMAC webhook signature verification: publishers sign events with shared secret, engine verifies X-Signature header

✓ Create signature verification middleware: compute HMAC-SHA256 of request body, compare with header, reject if mismatch

✓ Implement API key authentication for all publish/management endpoints

✓ API key management: generate secure keys (crypto.randomBytes), store hashed (bcrypt), support key rotation

✓ Per-tenant API keys: each tenant gets unique keys, key identifies tenant automatically

✓ Install @nestjs/throttler: rate limit publish endpoint (100 req/min per API key), management endpoints (30 req/min)

✓ Input validation: enforce max payload size (1MB), sanitize template variables against XSS

✓ Add Helmet middleware for security headers (CSP, HSTS, X-Frame-Options)

✓ Implement request logging with sanitized payloads (never log API keys, tokens, PII)

✓ Security test: attempt to publish without signature → verify 401, attempt with tampered body → verify 403

■ 10/10 DIFFERENTIATOR

HMAC signature verification proves you understand **zero-trust architecture**. Any service can claim to be legitimate — signatures cryptographically prove it. This is how Stripe, GitHub, and Twilio secure their webhooks.

Step 20 — Data Protection & Encryption

■ TOPICS TO STUDY FIRST

■ **Encryption at Rest:** Field-level encryption, MongoDB CSFLE, KMS concepts, envelope encryption

■ **PII Handling:** Identifying PII (email, phone, name), minimizing PII storage, data retention policies

■ **Secret Management:** Environment variables, Docker secrets, HashiCorp Vault concepts, secret rotation

■ **GDPR Basics:** Right to erasure, data portability, consent tracking, data processing records

■ **Audit Logging:** Who did what, when, immutable audit trail, tamper-evident logs

■ WHAT TO BUILD

✓ Implement field-level encryption for PII: encrypt email addresses and phone numbers in MongoDB using AES-256-GCM

✓ Create an EncryptionService: encrypt(plaintext) → { ciphertext, iv, tag }, decrypt(ciphertext, iv, tag) → plaintext

✓ Encrypt PII before storage, decrypt only when needed for delivery — minimize plaintext exposure

✓ Implement data retention policy: auto-delete notification records older than 90 days (configurable per tenant)

✓ Add GDPR data export endpoint: GET /api/v1/users/:userId/data-export → returns all user data as JSON

✓ Add GDPR data deletion endpoint: DELETE /api/v1/users/:userId/data → anonymize all records

✓ Store encryption keys separately from data (environment variable or KMS reference, never in code)

✓ Implement immutable audit log: every preference change, every data access, every admin action logged with timestamp + actor

✓ Add secret rotation support: API keys + encryption keys can be rotated without downtime

✓ Document security model in README: what's encrypted, what's logged, what's redacted

PHASE 12: Multi-Tenancy ■

■ 10/10 ADDITION — This phase elevates the project beyond typical portfolio projects

Step 21 — Multi-Tenant Architecture

■ TOPICS TO STUDY FIRST

■ **Multi-Tenancy Models:** Shared database (row-level isolation) vs database-per-tenant, pros/cons of each

■ **Tenant Isolation:** Data isolation, query scoping, tenant context propagation, cross-tenant leak prevention

■ **Tenant-Aware Middleware:** Extracting tenantId from API key/JWT/header, setting tenant context for request

■ **Resource Limits:** Per-tenant rate limits, per-tenant queue priorities, per-tenant template namespacing

■ **Onboarding:** Tenant registration, default configuration, API key provisioning

■ WHAT TO BUILD

✓ Choose shared-database model with row-level isolation (tenantId on every document)

✓ Create TenantMiddleware: extract tenantId from API key or JWT, attach to request context (AsyncLocalStorage)

✓ Create TenantGuard: verify tenantId exists and is valid on every request

✓ Scope all MongoDB queries to include tenantId automatically (Mongoose query middleware / plugin)

✓ Tenant-scoped templates: templates belong to tenants, shared 'system' templates available to all

✓ Tenant-scoped preferences: user preferences are per-tenant

✓ Per-tenant rate limits: configurable notification limits per tenant (stored in tenant config)

✓ Per-tenant provider configuration: Tenant A uses SendGrid, Tenant B uses SES

✓ Tenant management API: POST /api/v1/tenants (create), GET /api/v1/tenants/:id, PATCH (update config)

✓ Tenant onboarding flow: create tenant → generate API keys → seed default templates → return credentials

✓ Cross-tenant isolation test: verify Tenant A cannot access Tenant B's notifications, templates, or preferences

■ 10/10 DIFFERENTIATOR

Multi-tenancy transforms this from 'a notification service' into 'a notification platform.' It shows you can build **infrastructure that other teams/companies can use** — the exact thinking expected of senior/staff engineers.

PHASE 13: Load Testing & Benchmarks ■

■ 10/10 ADDITION — This phase elevates the project beyond typical portfolio projects

Step 22 — Load Testing with k6 & Documented Results

■ TOPICS TO STUDY FIRST

■ **Load Testing Concepts:** Throughput (RPS), latency (p50/p95/p99), saturation, error rate under load

■ **k6 Framework:** Test scripts in JavaScript, virtual users, stages (ramp-up, steady, ramp-down), thresholds, checks

■ **Test Scenarios:** Smoke test, load test, stress test, spike test, soak test — what each reveals

■ **Bottleneck Analysis:** CPU vs memory vs I/O vs network, MongoDB slow queries, Redis memory, queue depth growth

■ **Performance Tuning:** Connection pooling, query optimization, caching effectiveness, worker concurrency

■ WHAT TO BUILD

✓ Install k6, create test scripts in /load-tests directory

✓ Smoke test: 1 VU, 10 requests — verify system works under minimal load

✓ Load test: ramp from 10→100 VUs over 5 min, sustain 100 VUs for 10 min, measure throughput + latency

✓ Stress test: ramp to 500 VUs — find the breaking point, document where system degrades

✓ Spike test: jump from 10→300 VUs instantly — verify system handles traffic bursts (flash sale scenario)

✓ Test event publishing: POST /api/v1/events/publish with realistic payloads at scale

✓ Test notification feed: GET /api/v1/notifications with concurrent reads while writes happen

✓ Define pass/fail thresholds: p95 latency < 500ms, error rate < 1%, throughput > 200 RPS

✓ Run tests against Docker Compose environment, collect Prometheus metrics during load

✓ Document results in README 'Performance' section with actual numbers: 'Handles 500 events/second with p95 delivery latency under 3s on a single Docker Compose instance'

✓ Create a performance comparison table: throughput vs latency vs error rate at 50/100/200/500 VUs

✓ Identify and document bottlenecks found + how you optimized (e.g., added index, increased pool size, tuned concurrency)

■ 10/10 DIFFERENTIATOR

Nothing impresses reviewers like **real benchmark numbers**. 'Handles 500 events/second with p95 under 3s' backed by k6 results in the README proves you don't just build features — you validate they work under pressure.

PHASE 14: Schema Registry & Contract Testing ■

■ 10/10 ADDITION — This phase elevates the project beyond typical portfolio projects

Step 23 — Schema Registry & Event Contract Validation

■ TOPICS TO STUDY FIRST

■ **Schema Registry:** Centralized schema storage, schema evolution rules, compatibility modes (backward, forward, full)

■ **JSON Schema:** Draft-07, type definitions, required fields, additionalProperties, $ref for reuse

■ **Contract Testing:** Consumer-driven contracts, Pact framework concepts, provider verification

■ **Schema Evolution Rules:** Additive-only (add optional fields), never remove required fields, never change types, deprecation strategy

■ **Versioning Strategy:** Semantic versioning for schemas, version negotiation, multi-version support

■ WHAT TO BUILD

✓ Create an EventSchema collection in MongoDB: { eventType, version, jsonSchema, status (active/deprecated), createdAt }

✓ Build SchemaRegistryService: registerSchema(), getSchema(type, version), validateEvent(event), listSchemas()

✓ Store JSON Schema definitions for each event type (order.placed v1, payment.failed v1, user.signup v1)

✓ Implement schema validation in EventConsumerService: reject events that don't match their declared schema version

✓ Implement compatibility checking: when registering a new schema version, verify it's backward-compatible with previous version

✓ Schema management API: POST /api/v1/schemas (register), GET /api/v1/schemas/:eventType/versions, PUT (deprecate)

✓ Auto-reject events with deprecated schema versions (return 422 with migration guide)

✓ Create a schema migration guide document: how publishers should upgrade from v1→v2 of an event

✓ Write contract tests: for each event type, verify the engine correctly processes valid events and rejects invalid ones

✓ Add schema validation metrics: events_validated_total{result=pass|fail|deprecated}, schema_versions_active gauge

■ 10/10 DIFFERENTIATOR

Schema registries are what **Netflix, Uber, and Confluent** use to manage event contracts across hundreds of services. Implementing one — even lightweight — shows you understand the reality of evolving distributed systems.

PHASE 15: Cloud Deployment & IaC ■

■ 10/10 ADDITION — This phase elevates the project beyond typical portfolio projects

Step 24 — Kubernetes Manifests & Infrastructure-as-Code

■ TOPICS TO STUDY FIRST

■ **Kubernetes Basics:** Pods, Deployments, Services, ConfigMaps, Secrets, Namespaces, health probes

■ **K8s for Node.js:** Resource limits, readiness/liveness probes, graceful shutdown, HPA (auto-scaling)

■ **Helm Charts:** Chart structure, values.yaml, templating, chart dependencies, release management

■ **Infrastructure-as-Code:** Terraform basics, providers, resources, state management, modules

■ **Cloud Services:** AWS ECS/EKS or DigitalOcean K8s, managed MongoDB (Atlas), managed Redis (ElastiCache/Upstash)

■ WHAT TO BUILD

✓ Create /k8s directory with Kubernetes manifests for all services

✓ Deployment manifest: replicas=3, resource limits (256Mi-512Mi memory, 250m-500m CPU), rolling update strategy

✓ Readiness probe: GET /health (check DB + Redis + RabbitMQ connectivity), Liveness probe: GET /health/live (process alive)

✓ ConfigMap for non-secret config, Secret for API keys + DB credentials + encryption keys

✓ Service manifest: ClusterIP for internal services, LoadBalancer/Ingress for API endpoint

✓ HorizontalPodAutoscaler: scale 3→10 pods based on CPU > 70% or custom queue_depth metric

✓ Create a basic Helm chart wrapping the K8s manifests with configurable values.yaml

✓ Create /terraform directory with basic cloud infrastructure: VPC, ECS/K8s cluster, managed database, Redis

✓ Document two deployment paths in README: 'Quick Start (Docker Compose)' and 'Production (Kubernetes)'

✓ Add deployment architecture diagram showing cloud components and networking

■ PRO TIP

You don't need to actually deploy to AWS to get credit. Having **well-structured K8s manifests and Terraform files** in the repo demonstrates you understand production deployment — reviewers can read the configs and evaluate your infrastructure thinking.

PHASE 16: Documentation & Architecture Decision Records ■

■ 10/10 ADDITION — This phase elevates the project beyond typical portfolio projects

Step 25 — README, ADRs & System Design Document

■ TOPICS TO STUDY FIRST

■ **README Best Practices:** Badges, clear description, architecture diagram, quick start, API docs link, screenshots

■ **Architecture Decision Records (ADRs):** Lightweight docs capturing Context → Decision → Consequences for key choices

■ **System Design Document:** Problem statement, requirements, high-level design, detailed design, trade-offs, scalability analysis

■ **API Documentation:** OpenAPI/Swagger with @nestjs/swagger, request/response examples, error codes

■ **Diagramming:** Mermaid, draw.io, C4 model for architecture diagrams at different zoom levels

■ WHAT TO BUILD

✓ Polish README with: badges (CI, coverage, license), one-line description, architecture diagram (Mermaid), features list, quick start guide, API reference link, tech stack table, performance section with benchmark numbers

✓ Create /docs/adr/ directory with 5+ Architecture Decision Records:

✓ ADR-001: Why RabbitMQ + BullMQ instead of Kafka alone

✓ ADR-002: Why MongoDB over PostgreSQL for this use case

✓ ADR-003: Why per-channel queues over single queue with routing tags

✓ ADR-004: Why opossum circuit breaker over custom implementation

✓ ADR-005: Why shared-database multi-tenancy over database-per-tenant

✓ ADR-006: Why Pino over Winston for logging

✓ Write /docs/SYSTEM_DESIGN.md: problem statement, functional/non-functional requirements, capacity estimation (1M notifications/day = ~12 RPS average, 100 RPS peak), high-level architecture, component deep dives, data model, failure modes, scalability plan

✓ Install @nestjs/swagger, add decorators to all DTOs and controllers, expose Swagger UI at /api/docs

✓ Create Postman/Bruno collection with example requests for every endpoint

✓ Add CONTRIBUTING.md: coding standards, PR process, testing requirements, commit conventions

✓ Create an architecture diagram at 3 zoom levels: C1 (system context), C2 (containers), C3 (components)

■ 10/10 DIFFERENTIATOR

ADRs are your **interview cheat sheet** built into the repo. When asked 'why did you choose X over Y?', you have a documented, thoughtful answer. The system design doc shows you can think at the architectural level — not just code features.

Appendix A: 10/10 Project Description (for CV/Portfolio)

Use this updated description on your CV and portfolio:

Event-Driven Notification Engine — *NestJS, TypeScript, RabbitMQ, BullMQ, MongoDB, Redis, Socket.io, Twilio, Docker, Kubernetes, k6*

Architected a production-grade, multi-tenant, event-driven notification microservice that decouples notification logic from core business services. Processes domain events (order.placed, payment.failed, user.signup) and orchestrates multi-channel delivery across email, SMS, push, and in-app notifications.

Built a scalable multi-channel delivery system using isolated BullMQ queues per channel with fault isolation, exponential backoff with jitter, and circuit breaker patterns (opossum) with fallback provider routing to prevent cascading failures.

Implemented HMAC-SHA256 webhook signature verification for event authentication, field-level AES-256-GCM encryption for PII at rest, and API key authentication with per-tenant isolation. Added GDPR-compliant data export and deletion endpoints.

Designed idempotent event processing with two-tier deduplication (Redis + MongoDB), a lightweight schema registry with backward-compatibility validation, and strict event versioning to ensure stability across evolving contracts.

Built a template engine with Redis caching (1-hour TTL, stampede prevention), a user preference engine with timezone-aware quiet hours and priority overrides, and a distributed digest/batching system with Redis-based distributed locking.

Achieved 85%+ test coverage with unit, integration (mongodb-memory-server), and E2E tests. CI/CD pipeline via GitHub Actions (lint → typecheck → test → build → Docker push). Load tested with k6: handles 500+ events/second with p95 delivery latency under 3 seconds.

Full observability: structured logging (Pino) with correlation IDs via AsyncLocalStorage, Prometheus metrics (counters, histograms, gauges), Grafana dashboards, and real-time analytics via Server-Sent Events. Kubernetes-ready with Helm charts and Terraform IaC.

Appendix B: Recommended Learning Resources

NestJS

- Official Documentation — docs.nestjs.com
- NestJS official courses
- NestJS Microservices section

TypeScript

- TypeScript Handbook — typescriptlang.org
- Total TypeScript by Matt Pocock

Message Brokers

- RabbitMQ Tutorials (1-5) — rabbitmq.com/tutorials
- BullMQ Docs — docs.bullmq.io
- Enterprise Integration Patterns (Hohpe & Woolf)

System Design

- Designing Data-Intensive Applications — Martin Kleppmann
- System Design Interview — Alex Xu
- Martin Fowler's blog — Circuit Breaker, Event Sourcing

Testing

- Jest Documentation — jestjs.io
- NestJS Testing Guide in official docs
- k6 Documentation — k6.io/docs

Security

- OWASP Node.js Security Cheat Sheet
- Node.js Security Best Practices — nodejs.org/en/docs/guides/security

Docker & Kubernetes

- Docker Docs — docs.docker.com/get-started
- Kubernetes Basics — kubernetes.io/docs/tutorials
- 12-Factor App — 12factor.net

Monitoring

- Prometheus — prometheus.io/docs
- Grafana tutorials — grafana.com/tutorials
- OpenTelemetry — opentelemetry.io

■ FINAL WORDS

This 10/10 version covers everything: core architecture, resilience, security, multi-tenancy, testing, load benchmarks, CI/CD, cloud deployment, and architectural documentation. Each addition targets a specific gap that senior engineers and hiring managers look for. Build it step by step, understand the **why** behind every pattern, and document your decisions in ADRs.

You've got this, Mohamed. Build something that makes interviewers say 'this person thinks like a staff engineer.' ■